

2. 제어문



- 2-1 if문
- 2-2 while문
- 2-3 for문
- 2장 실습 과제

2-1. if문



■ if문의 기본 구조

```
if 조건A:  
    조건A가 참일 때 수행  
    ...  
elif 조건B:    # else if  
    조건B가 참일 때 수행  
    ...  
elif 조건C:  
    조건C가 참일 때 수행  
    ...  
else:  
    모든 조건이 거짓 일 때 수행  
    ...
```

if	필수. 한 번만
elif	선택. 여러 개 가능
else	선택. 한 번만

■ if문의 기본 구조

- 들여쓰기는 일반적으로 공백 문자(white space) 4개를 사용

<올바른 예>

```
if 조건문:  
    수행할_문장1  
    수행할_문장2  
    ...  
else:  
    수행할_문장A  
    수행할_문장B  
    ...
```

<잘못된 예>

```
if 조건문:  
    수행할_문장1  
수행할_문장2  
    ...  
else:  
    수행할_문장A  
        수행할_문장B  
    ...
```

2-1. if문



■ 비교 연산자

$x < y$	x가 y보다 작다.
$x > y$	x가 y보다 크다.
$x == y$	x와 y가 같다.
$x != y$	x와 y가 같지 않다.
$x >= y$	x가 y보다 크거나 같다.
$x <= y$	x가 y보다 작거나 같다.

```
x = 5
y = 3
print(x > y)      # True

print(x < y)      # False

print(x == y)     # False

print(x != y)     # True
```

2-1. if문

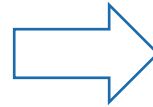


■ 비교 연산자의 연쇄 사용

```
x = 0.6
```

```
# x가 0보다 크고 1보다 작은가?  
(0 < x) and (x < 1)
```

```
# x가 0 이상 0.5 이하인가?  
(0 <= x) and (x <= 0.5)
```



```
x = 0.6
```

```
# x가 0보다 크고 1보다 작은가?  
0 < x < 1      # True
```

```
# x가 0 이상 0.5 이하인가?  
0 <= x <= 0.5  # False
```

2-1. if문



■ 논리 연산자

x or y	x와 y 둘 중 하나만 참이어도 참이다.
x and y	x와 y 모두 참이어야 참이다.
not x	x가 거짓이면 참이다.

```
x = True  
y = False
```

```
print(x or y)    # True
```

```
print(x and y)   # False
```

```
print(not x)     # False
```

2-1. if문



■ 관계 연산자 in, not in

in	not in
x in 리스트	x not in 리스트
x in 튜플	x not in 튜플
x in 문자열	x not in 문자열

```
1 in [1, 2, 3]      # True
```

```
1 not in [1, 2, 3]  # False
```

```
'd' in ('a', 'b', 'c')  # False
```

```
'k' not in 'postech'    # True
```

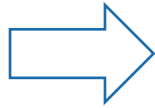

■ 조건부 표현식

- 기존 if-else 문을 한 줄의 표현식으로 간단히 작성하는 방법

```
score = 95

if score >= 80:
    result = "합격"
else:
    result = "불합격"

print(result)
```



```
score = 95

result = "합격" if score >= 80 else "불합격"

print(result)
```

■ 연습 1

- 만약 6000원 이상의 돈을 가지고 있으면 두쫂꾸를 사먹고, 그렇지 않으면 먹지 마라.

```
money = 5000

if money >= 6000:
    print("두쫂꾸를 사먹어라")
else:
    print("먹지 마라")
```

■ 연습 2

- 돈이 6000원 이상 있거나 카드가 있다면 두쫘꾸를 사먹고, 그렇지 않으면 먹지 마라.

```
money = 5000
card = True

if money >= 6000 or card:
    print("두쫘꾸를 사먹어라")
else:
    print("먹지 마라")
```



■ 연습 3

- 만약 주머니에 지갑이 있으면 두쫌쿠를 사먹고, 그렇지 않으면 먹지 마라.

```
pocket = ['cellphone', 'earphones', 'wallet']  
  
if 'wallet' in pocket:  
    print("두쫌쿠를 사먹어라")  
else:  
    print("먹지 마라")
```

■ 연습 4

- 만약 주머니에 지갑이 있으면 가만히 있고, 없으면 지갑을 찾아라.

```
pocket = ['cellphone', 'earphones', 'wallet']

if 'wallet' in pocket:
    pass      # 아무 일도 하지 않음
else:
    print("지갑을 찾아라")
```



■ 연습 5

- 만약 지갑에 현금이 있으면 두꺼비를 사고, 카드만 있으면 카드로 사고, 둘 다 없으면 사지 마라.

```
wallet = ['cash', 'card']

if 'cash' in wallet:
    print("두꺼비를 사라")
elif 'card' in wallet:
    print("카드로 두꺼비를 사라")
else:
    print("사지 마라")
```

■ 예제 1. 택배 요금 계산 프로그램

- 다음 조건에 따라 최종 택배 요금을 계산하여 출력하시오.

- 무게에 따라 기본 요금을 먼저 결정한다.
 - 5kg 이하 → 3000원
 - 10kg 이하 → 5000원
 - 10kg 초과 → 8000원
- 이후 추가 조건에 따라 요금을 더한다.
 - 도서산간이면 2000원 추가
 - 파손주의이면 1000원 추가
- 단, 도서산간이면서 10kg 초과이고 파손주의인 경우에는 접수할 수 없다.

■ 조건 예시

```
weight = 8          # kg
is_remote = True     # 도서산간 여부
is_fragile = False   # 파손주의 여부

price = 0
```

■ 예제 1. 모범 답안

```
# 조건 바뀌가면서 실험
```

```
weight = 8
```

```
is_remote = True
```

```
is_fragile = False
```

```
price = 0
```

```
# 1단계: 접수 가능 여부 판단
```

```
if is_remote and weight > 10 and is_fragile:  
    print("접수 불가")
```

```
else:
```

```
# 2단계: 가격 계산
```

```
if weight <= 5:
```

```
    price = 3000
```

```
elif weight <= 10:
```

```
    price = 5000
```

```
else:
```

```
    price = 8000
```

```
if is_remote:
```

```
    price += 2000
```

```
if is_fragile:
```

```
    price += 1000
```

```
print("최종 요금:", price)
```


2-2. while문



■ while문의 기본 구조

```
while 조건문:  
    조건이 참일 동안 수행할_문장1  
    수행할_문장2  
    수행할_문장3  
    ...
```

2-2. while문



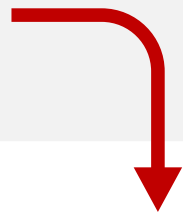
■ break문

- break문을 사용하면 반복문을 즉시 종료하고 밖으로 빠져나간다.

```
num = 0

while True:
    num += 1
    print(num)

    if num == 5:
        break
```



1
2
3
4
5

2-2. while문

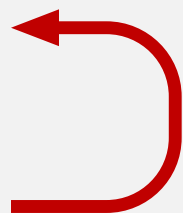


■ continue문

- continue문을 사용하면 현재 반복을 건너뛰고 다음 반복을 실행한다.

```
num = 0

while num < 5:
    num += 1
    if num == 3:
        continue
    print(num)
```



1
2
4
5

2-2. while문



■ 무한루프

- 조건이 항상 참이어서 반복문이 끝나지 않는 상태

```
while True:  
    print("Ctrl+C를 눌러 종료")
```

주피터 노트북 셀에서는 중단 버튼을 눌러 종료

2-2. while문



■ while-else문

- while 문이 정상적으로 종료되었을 때 else 절이 실행된다.

<정상 종료>

```
count = 0
while count < 3:
    print(count)
    count += 1
else:
    print("정상 종료")
```

<break로 인한 종료>

```
count = 0
while count < 3:
    if count == 2:
        break
    print(count)
    count += 1
else: # 실행되지 않음
    print("정상 종료")
```

2-2. while문



■ 중첩된 while문

- while 문 안에 또 다른 while 문을 중첩하여 사용할 수 있다.

```
i = 1                                # (i, j)

while i <= 3:                        # (1, 1)
    j = 1                            # (2, 1) (2, 2)
    while j <= i:                    # (3, 1) (3, 2) (3, 3)
        print("*", end="")
        j += 1
    print()
    i += 1
```

2-2. while문



■ 연습 7

- 물의 온도가 100도가 될 때까지 10도씩 계속 가열한다.

```
temperature = 20

while temperature < 100:
    temperature += 10    # temperature = temperature + 10
    print("현재 온도:", temperature)

print("물이 끓는다")
```

2-2. while문



■ 연습 8

- 사용자에게 선택지를 입력 받아 4번을 선택하면 프로그램을 종료한다.

```
prompt = """
1. One
2. Two
3. Three
4. Quit

Enter number: """
```

```
number = 0

while number != 4:
    print(prompt)
    number = int(input())

print("프로그램 종료")
```


■ 예제 2. 커피 자판기 프로그램 (재고 3잔)

- 사용자로부터 돈을 입력받아 다음과 같이 동작하도록 프로그램을 작성하시오.
 - 커피 한 잔의 가격은 300원이다.
 - 입력한 금액이 가격보다 적으면 커피를 제공하지 않는다.
 - 가격 이상을 입력하면 커피를 한 잔 제공한다.
 - 입력 금액이 가격보다 많으면 거스름돈을 출력한다.
 - 커피가 모두 소진되면 판매를 중지한다.
 - 커피가 모두 소진될 때까지 위 과정을 반복한다.

```
coffee = 3
while True:
    money = int(input("돈을 넣어 주세요: "))
```

이후 코드 작성

2-2. while문



■ 예제 2. 모범 답안

```
coffee = 3
while True:
    money = int(input("돈을 넣어 주세요: "))

    if money < 300:
        print("돈을 돌려줍니다.")
        continue

    print("커피를 줍니다.")
    coffee -= 1

    if money > 300:
        print("거스름돈:", (money - 300))

    print("남은 커피:", coffee)

    if coffee == 0:
        print("커피가 다 떨어졌습니다. 판매를 중지합니다.")
        break
```

■ for문의 기본 구조

```
for 변수 in 리스트(또는 튜플, 문자열):  
    반복 수행할 문장1  
    반복 수행할 문장2  
    ...
```

2-3. for문



■ for문의 기본 사용

```
steps = ["first", "second", "third"]
```

```
for step in steps:  
    print(step)
```

```
first  
second  
third
```

```
pairs = [(1,2), (2,4), (3,6)]
```

```
for a, b in pairs:  
    print(a + b)
```

```
3  
6  
9
```



■ range 함수

- range() 함수는 연속된 숫자를 만들어 주어 for문과 함께 자주 사용된다.

```
# 1부터 10까지 더하기  
  
add = 0  
  
for i in range(1, 11):    # 10이상 11 미만  
    add = add + i  
  
print(add)                # 55
```

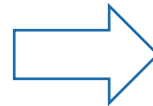
■ list comprehension

- 리스트 컴프리헨션은 리스트를 생성할 때 for문과 if문을 한 줄로 표현할 수 있는 문법이다.

```
nums = [1, 2, 3, 4, 5]
result = []

for n in nums:
    result.append(n + 1)

print(result)
```



```
nums = [1, 2, 3, 4, 5]
result = [n + 1 for n in nums]
print(result)
```

■ break문, for-else문

- for-else문에서 else절은 for문이 break 없이 정상적으로 종료되었을 때만 실행된다.

<정상 종료>

```
for i in range(1, 6):  
    print(i)  
else:  
    print("모든 숫자를 출력했습니다.")
```

<break로 인한 종료>

```
for i in range(1, 6):  
    if i == 4:  
        break  
    print(i)  
else:  
    print("모든 숫자를 출력했습니다.")
```



■ enumerate 함수

- enumerate() 함수는 리스트의 값과 함께 번호를 동시에 얻고 싶을 때 활용한다.

```
colors = ['red', 'green', 'blue']
```

```
for num, color in enumerate(colors, 1):    # 1부터 시작. 없으면 0부터  
    print(f"{num}번째 색상: {color}")
```

```
1번째 색상: red  
2번째 색상: green  
3번째 색상: blue
```


■ zip 함수

- zip() 함수는 여러 리스트의 같은 위치에 있는 값들을 묶어서 동시에 순회할 때 사용한다.

```
days = ['월', '화', '수']  
menus = ['김밥', '라면', '돈까스']  
  
for day, menu in zip(days, menus):  
    print(f"{day}요일 점심: {menu}")
```

```
월요일 점심: 김밥  
화요일 점심: 라면  
수요일 점심: 돈까스
```

■ 연습 9

- 총 5개의 과제에 대해 점수가 3점 이상이면 통과, 아니면 미통과를 출력하시오.

```
# tasks1.py
scores = [5, 2, 4, 1, 3]  # 과제 점수 리스트

number = 0  # 과제 번호
for score in scores:
    number = number + 1
    if score >= 3:
        print("%d번 과제는 통과입니다." % number)
    else:
        print("%d번 과제는 미통과입니다." % number)
```

■ 예제 3

- continue문을 사용하여 3점 이상인 과제에 대해서만 통과 메시지를 출력하시오.

```
# tasks2.py  
scores = [5, 2, 4, 1, 3]  # 과제 점수 리스트
```

■ 예제 3. 모범 답안

- continue문을 사용하여 3점 이상인 과제에 대해서만 통과 메시지를 출력하시오.

```
# tasks2.py
scores = [5, 2, 4, 1, 3]  # 과제 점수 리스트

number = 0  # 과제 번호
for score in scores:
    number = number + 1
    if score < 3:
        continue
    print("%d번 과제는 통과입니다." % number)
```

■ 예제 4

- range() 함수와 리스트 인덱싱을 사용하여 예제 3과 같은 결과를 출력하시오.

```
# tasks3.py  
scores = [5, 2, 4, 1, 3]  # 과제 점수 리스트
```

■ 예제 4. 모범 답안

- range() 함수와 리스트 인덱싱을 사용하여 예제 3과 같은 결과를 출력하시오.

```
# tasks3.py
scores = [5, 2, 4, 1, 3]  # 과제 점수 리스트

for n in range(len(scores)):
    if scores[n] < 3:
        continue
    print("%d번 과제는 통과입니다." % (n+1))
```

■ 연습 11

- 리스트 컴프리헨션을 사용하여 리스트에서 3 이상인 값만 선택하여 새로운 리스트를 만드시오.

[표현식 for 항목 in 반복_가능_객체 if 조건문]

```
nums = [1, 2, 3, 4, 5]
result = [n for n in nums if n >= 3]

print(result)
```

■ 예제 5. 하루 일정 처리하기

- 다음 조건에 따라 for-break문을 사용하여 하루 일정을 출력하시오.
 1. 일정에 번호를 붙여 순서대로 출력하시오.
 2. "점심"이 나오면 번호 없이 "--- 점심시간 ---"을 출력하시오.
 3. "퇴근"이 나오면 "업무 종료"를 출력하고 반복을 종료하시오.

```
schedule = ["회의", "자료 정리", "점심", "발표 준비", "퇴근", "운동", "독서"]
```

■ 출력 예시

```
1. 회의
2. 자료 정리
--- 점심시간 ---
4. 발표 준비
5. 퇴근
   업무 종료
```


■ 예제 5. 모범 답안

```
schedule = ["회의", "자료 정리", "점심", "발표 준비", "퇴근", "운동", "독서"]

for i, task in enumerate(schedule, start=1):
    # 점심 시간 별도 출력
    if task == "점심":
        print("--- 점심시간 ---")
        continue

    # 일반 일정은 번호와 함께 출력
    print(f"{i}. {task}")

    # 퇴근이 나오면 반복 종료
    if task == "퇴근":
        print("업무 종료")
        break
```